

Basket-AR: etichettatura e riconoscimento di azioni nella pallacanestro

Progetto di Fondamenti di Visione Artificiale e Biometria

Gruppo 10
Giovanni Paolo Chierchia
Sebastiano Caliendo
Università degli Studi di Salerno

Anno Accademico 2025/2026

Repository GitHub:

<https://github.com/Giop378/FVAB-Riconoscimento-azioni-basket>

Abstract

Questo lavoro presenta una pipeline modulare per il riconoscimento automatico di azioni in video RGB di partite di basket. L'obiettivo è duplice: classificare clip annotate e applicare il modello a video lunghi, producendo un report temporale degli eventi rilevati. Il dataset custom utilizzato comprende sette classi finali, relative a passaggi e tiri distinti per tipologia ed esito, insieme a classi di background impiegate per rappresentare segmenti privi di azioni rilevanti. Il problema è affrontato combinando feature visuali DINOv3 pre-estratte, un classificatore temporale basato su Transformer e una formulazione gerarchica, che separa il riconoscimento delle macro-azioni dalla classificazione del tipo di tiro e dalla stima dell'esito. Nella configurazione finale, la rappresentazione visuale viene arricchita con feature geometriche e temporali ottenute dal tracking di palla e canestro tramite detector YOLO. I risultati sperimentali mostrano che l'approccio proposto è efficace nella classificazione delle clip annotate, in particolare nella distinzione tra azioni rilevanti e background. L'inferenza su video lunghi consente inoltre di generare annotazioni temporali degli eventi, pur evidenziando criticità nella localizzazione precisa e nel riconoscimento delle classi meno rappresentate.

1 Introduzione

Contesto e motivazione

Il **riconoscimento automatico di azioni sportive da video** rappresenta un problema rilevante nell'ambito della **visione artificiale**, con applicazioni nell'*analisi tattica*, nella produzione automatica di report e nel supporto alla valutazione delle prestazioni. Nel caso della pallacanestro, il compito risulta particolarmente complesso a causa della natura **rapida e dinamica** del gioco: le azioni possono avvenire in sequenza ravvicinata, coinvolgere più giocatori contemporaneamente ed essere influenzate da **occlusioni**, movimenti della camera e ridotta visibilità della palla.

Limiti dei lavori esistenti

Sebbene in letteratura siano presenti dataset e metodi dedicati al *basketball action recognition*, molti lavori si concentrano su insiemi limitati di azioni, su clip già segmentate o su scenari non direttamente sovrapponibili all'analisi di video completi di partita. Alcuni dataset distinguono azioni generali come *passaggio*, *passeggiato* o *tiro*, mentre altri si focalizzano soprattutto sulle conclusioni a canestro, senza considerare in modo congiunto la **localizzazione temporale degli eventi** all'interno di una sequenza continua [1, 2, 3]. Questo rende necessario progettare soluzioni adatte a **dati custom**, annotazioni granulari e condizioni di acquisizione realistiche.

Obiettivo del lavoro

In questo lavoro viene affrontato il problema del riconoscimento di azioni cestistiche a partire da video RGB di partite giovanili, acquisiti tramite una **camera automatica mobile**. L'obiettivo è riconoscere sette classi di interesse: **passaggio**, **tiroDaDue0**, **tiroDaDue1**, **tiroDaTre0**, **tiroDaTre1**, **tiroLibero0** e **tiroLibero1**, dove il suffisso indica l'esito del tiro. Le classi **idle** e **non-gioco** vengono invece utilizzate durante l'addestramento come esempi di *background*, ma non sono incluse nel report finale degli eventi.

Pipeline proposta

Per affrontare il problema è stata sviluppata una **pipeline modulare**, organizzata in due flussi principali: uno per la classificazione di clip già segmentate e uno per l'inferenza su video completi.

- **Pipeline su singole clip**: ogni clip annotata viene rappresentata tramite feature **DINOv3** estratte *frame by frame*; tali feature vengono poi elaborate da un **Temporal Transformer**. Nella configurazione finale, la classificazione è formulata in modo gerarchico: **L1** distingue tra **passaggio**, **tiro** e **no-action**; **L2** classifica la tipologia del tiro, distinguendo tra **tiroDaDue**, **tiroDaTre** e **tiroLibero**; **L3** stima l'esito, distinguendo tra **tiro0** e **tiro1**. Le predizioni di L2 e L3 vengono infine combinate per ricostruire la classe finale del tiro.
- **Pipeline su video completo**: il video viene analizzato tramite una strategia di **sliding window** multi-scala. Per ogni finestra vengono estratte o riutilizzate le feature visuali e le informazioni di tracking palla-canestro; successivamente, la finestra viene classificata dal modello gerarchico. Le predizioni locali vengono poi filtrate tramite **post-processing temporale**, scartando **no-action**, applicando soglie di confidenza, aggregando finestre vicine e rimuovendo eventi sovrapposti. L'output finale è una lista di eventi con label, tempo di inizio e tempo di fine.

In entrambe le pipeline, le feature visuali sono arricchite, nelle configurazioni finali, con informazioni **geometriche e temporali** derivate dal tracking di **ball** e **rim**, così da fornire al classificatore indizi espliciti sul movimento della palla e sulla sua relazione con il canestro.

Contributo del lavoro

Il contributo principale del lavoro consiste nella realizzazione di un sistema completo per il riconoscimento di azioni cestistiche annotate, dalla costruzione del dataset fino all'inferenza su video lunghi. La pipeline proposta non si limita alla classificazione di clip isolate, ma produce una lista di **eventi temporalmente localizzati**, associando a ciascuna azione una label e un intervallo temporale. Questa impostazione consente di valutare il modello sia a livello di clip, tramite metriche di classificazione, sia a livello di evento, verificando la capacità del sistema di generare annotazioni coerenti su sequenze video complete.

Organizzazione del paper

Il resto del paper è organizzato come segue. La Sezione 2 presenta i principali dataset e metodi di riferimento per l'*action recognition video-based* e per il dominio cestistico. La Sezione 3 descrive il dataset, l'estrazione delle feature, il classificatore gerarchico e la pipeline di inferenza su video lunghi. La Sezione 4 riporta gli esperimenti condotti e discute i risultati ottenuti. Infine, la Sezione 5 riassume il lavoro svolto e presenta possibili sviluppi futuri.

2 Stato dell'arte

2.1 Dataset e benchmark per il basketball action recognition

Benchmark esistenti

Nel campo del *basketball action recognition* sono stati proposti diversi dataset e benchmark, caratterizzati da livelli di **granularità**, modalità di acquisizione e obiettivi differenti. Un primo riferimento rilevante è **SpaceJam**, dataset specifico per il riconoscimento di azioni cestistiche, con clip RGB associate a classi come *walk*, *no action*, *run*, *defense*, *dribble*, *ball in hand*, *pass*, *block*, *pick* e *shoot*. Il dataset include anche coordinate articolari dei giocatori, utilizzabili come rappresentazione scheletrica, ma non distingue in modo fine il tipo di tiro né il relativo esito [1].

Un secondo benchmark vicino al nostro scenario è **Basketball-51**, costruito a partire da partite NBA e focalizzato principalmente sulle **azioni di tiro**. Le sue classi distinguono diverse tipologie di conclusione, come tiri da due, tiri da tre, tiri liberi e tiri dalla media distanza, separando anche **esito positivo** e **esito negativo**. Tuttavia, il dataset non copre altre dinamiche di gioco rilevanti, come il passaggio, e non affronta direttamente il problema dell'**inferenza continua su video completi** [2].

Un dataset più recente è **MultiSubjects**, pensato per il riconoscimento di azioni di basket in scenari reali di palestra. Esso raccoglie video di soggetti diversi, acquisiti da più punti di vista, e fornisce annotazioni spaziali tramite *bounding box* e informazioni scheletriche basate su *keypoint frame-wise*. Il principale contributo del dataset è l'attenzione alla **variabilità dei soggetti** e delle condizioni di acquisizione; tuttavia, considera solo tre macro-azioni, cioè *dribbling*, *layup* e *shooting*, risultando meno adatto a un riconoscimento fine degli eventi di partita [3].

Relazione con il nostro dataset

Questi benchmark evidenziano la disponibilità di risorse utili, ma non perfettamente allineate al nostro obiettivo. Per questo motivo, nel progetto è stato costruito un **dataset custom** a partire da clip annotate manualmente, organizzato in nove classi: **passaggio**, **tiroDaDue0**, **tiroDaDue1**, **tiroDaTre0**, **tiroDaTre1**, **tiroLibero0**, **tiroLibero1**, **idle** e **non-gioco**. Le prime sette rappresentano le **azioni di interesse**, mentre **idle** e **non-gioco** sono classi ausiliarie utilizzate per riconoscere finestre temporali prive di eventi rilevanti.

2.2 Metodi per action recognition video-based

Modellazione spaziale e temporale

Il riconoscimento automatico di azioni da video viene affrontato in letteratura attraverso diverse famiglie di metodi, che si distinguono principalmente per il modo in cui modellano l'**informazione spaziale** dei frame e la sua evoluzione temporale. A differenza della classificazione di immagini statiche, l'*action recognition* richiede infatti di considerare la **dinamica dell'azione nel tempo**.

Modelli video end-to-end

Una prima classe di approcci è costituita dai modelli video *end-to-end* basati su convoluzioni spazio-temporali, come **C3D**, **I3D**, **R3D**, **R(2+1)D** e **SlowFast**. Queste architetture elaborano direttamente sequenze di frame e apprendono congiuntamente feature spaziali e temporali. Il loro vantaggio principale è la capacità di modellare esplicitamente il movimento; tuttavia, richiedono generalmente **dataset ampi**, risorse computazionali elevate e un numero significativo di esempi per generalizzare correttamente [4, 5, 6, 7, 8].

Transformer video

Accanto ai modelli convoluzionali, sono stati proposti approcci basati su attenzione e **Transformer video**, come **ViViT**, **TimeSformer** e **Video-Swin**. Questi metodi estendono il meccanismo

di *self-attention* al dominio video e permettono di catturare relazioni spazio-temporali anche a lungo raggio. Nonostante le prestazioni competitive, la loro **complessità computazionale** e il numero elevato di parametri possono rappresentare un limite in presenza di dataset ridotti o vincoli di efficienza [9, 10, 11].

Approcci skeleton-based

Un'altra famiglia di metodi è rappresentata dagli approcci *skeleton-based*, tra cui **ST-GCN** e successive architetture basate su grafi spazio-temporali. In questo caso l'azione viene descritta tramite le coordinate dei *keypoint* corporei, modellando le relazioni tra articolazioni e la loro evoluzione nel tempo. Tali metodi riducono la dipendenza da sfondo, colore e texture, ma la loro efficacia dipende fortemente dalla **qualità della stima della posa** [12].

Feature pre-addestrate e classificazione temporale

Una direzione alternativa consiste nell'utilizzare **rappresentazioni visive pre-addestrate** seguite da moduli di classificazione temporale. In questo schema, un backbone estrae feature dai frame o da brevi segmenti video, mentre un classificatore successivo modella la sequenza temporale e produce la predizione finale. Tale impostazione consente di separare la fase di **estrazione delle feature** da quella di addestramento del classificatore, riducendo il costo computazionale degli esperimenti e facilitando il riuso di rappresentazioni apprese su dataset di grandi dimensioni.

Tra i feature extractor utilizzabili in questo scenario rientrano sia backbone convoluzionali classici sia modelli visuali più recenti. Le reti **ResNet**, tra cui **ResNet18**, rappresentano una baseline leggera e consolidata grazie all'uso di connessioni residuali, che facilitano l'addestramento di reti profonde [13]. **ConvNeXt** propone invece una modernizzazione delle architetture convoluzionali, incorporando alcune scelte progettuali ispirate ai Vision Transformer pur mantenendo una struttura puramente convoluzionale; varianti compatte come **ConvNeXt-Tiny** risultano interessanti quando si cerca un compromesso tra qualità delle feature e costo computazionale [14].

Una famiglia più recente è costituita dai **foundation model visuali self-supervised**, come **DINOv2** e **DINOv3**. Questi modelli vengono pre-addestrati su grandi collezioni di immagini senza richiedere annotazioni manuali e producono rappresentazioni generali riutilizzabili in diversi task downstream [15, 16]. In uno scenario con dataset custom di dimensioni limitate, l'uso di **feature pre-addestrate** può quindi rappresentare una scelta efficace rispetto all'addestramento *end-to-end* di un grande modello video.

Nel panorama dei feature extractor alternativi rientrano anche architetture efficienti come **EfficientNet**, modelli *vision-language* come **CLIP**, e approcci *self-supervised* basati su *masked autoencoding*, come **MAE** e **VideoMAE** [17, 18, 19, 20]. Queste soluzioni mostrano come la qualità della rappresentazione visiva possa essere migliorata attraverso strategie diverse: scaling efficiente delle CNN, *pre-training* multimodale immagine-testo o ricostruzione di porzioni mascherate dell'input.

Considerazioni generali

Nel complesso, lo stato dell'arte mostra che l'*action recognition video-based* richiede modelli capaci di combinare **informazione spaziale** e **informazione temporale**. La scelta dell'approccio dipende però da fattori quali dimensione del dataset, disponibilità di risorse computazionali e obiettivo applicativo finale.

2.3 Metodi specifici per il basket

Criticità del dominio cestistico

Nel dominio della pallacanestro, l'*action recognition* presenta criticità specifiche: le azioni sono rapide, spesso sovrapposte e visivamente simili; inoltre, la presenza di più giocatori, occlusioni e variazioni di punto di vista rende difficile isolare l'evento rilevante. Un elemento particolarmente importante è il ruolo della **palla**, poiché azioni come passaggi e tiri richiedono di osservare non solo la postura del giocatore, ma anche l'**evoluzione temporale dell'azione** e le interazioni con il canestro.

Approcci utilizzati in letteratura

In letteratura, i metodi specifici per il basket includono approcci basati su sensori, approcci *video-based* e approcci *skeleton-based*. I metodi basati su sensori inerziali possono descrivere accuratamente il movimento del corpo, ma richiedono dispositivi indossabili e risultano poco adatti all'analisi di filmati RGB già acquisiti. Per questo motivo, nel nostro scenario sono più rilevanti i metodi *video-based* e *skeleton-based*, eventualmente integrati con informazioni temporali o contestuali.

Un lavoro recente particolarmente rilevante è **ACA-Net**, una rete progettata per il *basketball action recognition*. Il modello utilizza una backbone **ResNet50** arricchita con due moduli: **LSTA**, dedicato alla modellazione di dipendenze temporali a breve e lungo termine, e **TSCI**, orientato al miglioramento delle interazioni tra informazione spaziale e canali. Questa architettura nasce per distinguere azioni cestistiche visivamente simili e sfruttare meglio il **contesto temporale**. Gli esperimenti su **SpaceJam** e **Basketball-51** mostrano prestazioni elevate, confermando l'importanza di combinare feature discriminanti e modellazione temporale [21].

Anche dataset recenti come **MultiSubjects** evidenziano l'interesse verso scenari più realistici e verso l'uso di annotazioni spaziali e *keypoint* scheletrici. Tuttavia, i risultati mostrano che i metodi *skeleton-based* possono essere penalizzati da **occlusioni**, movimenti rapidi e condizioni di acquisizione non controllate [3].

Implicazioni per il nostro lavoro

Nel nostro lavoro, questi aspetti hanno motivato una soluzione modulare: **feature visive pre-addestrate**, classificazione temporale e, nelle configurazioni più avanzate, informazioni contestuali derivate dal **tracciamento di palla e canestro**. Questa scelta consente di sfruttare rappresentazioni robuste, contenere il costo computazionale e mantenere la possibilità di applicare il modello a finestre temporali estratte da video lunghi.

2.4 Limiti dei lavori esistenti rispetto al nostro scenario

Granularità delle annotazioni

Nonostante i dataset e i metodi presenti in letteratura costituiscano riferimenti utili, essi non risultano pienamente allineati allo scenario affrontato in questo progetto. Un primo limite riguarda la **granularità delle annotazioni**: alcuni dataset descrivono azioni cestistiche generali, come *pass*, *dribble* o *shoot*, senza distinguere il tipo di tiro e il relativo esito; altri, invece, si concentrano sui tiri ma non includono dinamiche di gioco come il passaggio.

Vincolo sulla lunghezza temporale

Un ulteriore aspetto riguarda la gestione della **durata delle clip**. Molti modelli video *end-to-end*, come **C3D**, **I3D**, **R3D**, **R(2+1)D**, **SlowFast** e diversi **Transformer video** come **ViViT**, **TimeSformer** e **Video-Swin**, vengono comunemente addestrati e valutati su clip campionate a una lunghezza temporale fissata o comunque standardizzata [4, 5, 6, 7, 8, 9, 10, 11]. Nel nostro scenario, invece, le clip annotate seguono la durata reale dell'azione e possono quindi avere

lunghezze diverse. Questo rende meno immediato l'utilizzo diretto di architetture video complete e motiva l'adozione di una pipeline modulare basata su feature *frame-level* e classificazione temporale.

Inferenza su video completi

Un secondo limite riguarda il contesto di utilizzo. Molti benchmark sono costruiti a partire da clip già segmentate o da video di partite professionistiche con eventi già isolati. Questo semplifica il compito rispetto all'applicazione su video lunghi, dove il modello deve analizzare sequenze continue contenenti anche fasi non rilevanti, interruzioni, occlusioni e azioni sovrapposte. Per il nostro obiettivo finale, quindi, non è sufficiente classificare clip pre-tagliate: è necessario produrre **eventi temporalmente localizzati** all'interno di un video completo.

Disponibilità dei dati e sbilanciamento

Un ulteriore vincolo è rappresentato dalla disponibilità dei dati. Il dataset utilizzato nel progetto è **custom**, costruito a partire da un numero limitato di video sorgente e caratterizzato da uno **sbilanciamento significativo tra le classi**. Questo rende meno adatti modelli video *end-to-end* molto complessi e favorisce soluzioni più modulari, capaci di riutilizzare feature pre-addestrate e limitare il costo degli esperimenti.

Necessità di informazioni contestuali

Infine, il solo contenuto visivo o scheletrico può non essere sufficiente per distinguere eventi simili, soprattutto nel caso dei tiri con esito diverso. Per questi motivi, il nostro scenario richiede una pipeline capace di combinare **informazione visiva**, **modellazione temporale** e, ove possibile, informazioni contestuali sulla scena.

2.5 Posizionamento del nostro approccio

Scelta di una pipeline modulare

Alla luce dei limiti evidenziati, il nostro approccio si colloca tra i metodi basati su **feature visive pre-addestrate** e classificazione temporale. Invece di addestrare da zero un modello video *end-to-end*, la pipeline separa l'estrazione delle rappresentazioni dalla fase di addestramento del classificatore. In fase sperimentale sono stati considerati diversi feature extractor, tra cui **ResNet18**, **ConvNeXt-Tiny**, **DINOv2** e **DINOv3**, privilegiando progressivamente le rappresentazioni più robuste per il dominio considerato. Questa scelta permette di ridurre il costo computazionale, riutilizzare feature già apprese e confrontare più configurazioni sperimentali in modo efficiente.

Applicazione a video lunghi

Il sistema è inoltre progettato per essere applicato non solo a clip isolate, ma anche a video completi tramite finestre temporali scorrevoli. Le predizioni ottenute sulle singole finestre possono poi essere filtrate e aggregate per produrre una lista di eventi con classe e timestamp. In questo modo, il metodo risponde al duplice obiettivo del progetto: riconoscere **azioni cestistiche annotate** con sufficiente granularità e mantenere una pipeline applicabile a sequenze video più lunghe.

Integrazione del tracking palla-canestro

Nelle configurazioni più avanzate, l'approccio integra anche informazioni legate al **tracciamento di palla e canestro**, con l'obiettivo di fornire al classificatore elementi contestuali utili per distinguere azioni visivamente simili. Questa impostazione rappresenta un compromesso tra **accuratezza**, semplicità implementativa ed efficienza computazionale, risultando adatta a un dataset custom di dimensioni limitate e fortemente sbilanciato.

3 Metodologia

3.1 Formulazione del problema

Obiettivo del sistema

L'obiettivo del progetto è sviluppare un sistema di *action recognition* per video di partite di basket, capace di riconoscere automaticamente le azioni annotate e di produrre, in fase di inferenza su video interi, un report con la classe dell'azione rilevata e il relativo intervallo temporale.

Il problema è reso complesso dalla natura dinamica del basket: le azioni sono **rapide**, possono avvenire in sequenza ravvicinata e sono spesso influenzate da **occlusioni**, distanza dalla camera e movimento della palla. Inoltre, i video sono acquisiti da una **singola camera mobile automatica**, progettata per seguire lo sviluppo dell'azione, ma comunque soggetta a errori di inquadratura, occlusioni e temporanee perdite di elementi rilevanti come palla, giocatori o canestro.

Classi da riconoscere

Le classi finali da riconoscere sono sette: **passaggio**, **tiroDaDue0**, **tiroDaDue1**, **tiroDaTre0**, **tiroDaTre1**, **tiroLibero0** e **tiroLibero1**. I tiri sono distinti sia in base alla **tipologia**, cioè tiro da due, tiro da tre o tiro libero, sia in base all'**esito**, dove il suffisso 1 indica un tiro segnato e il suffisso 0 un tiro fallito.

Durante l'addestramento sono inoltre presenti le classi **idle** e **non-gioco**, utilizzate come esempi di **background** per insegnare al modello a riconoscere i segmenti in cui non deve essere prodotta alcuna azione. Nel report finale, queste due classi non vengono mostrate, ma sono trattate come **no-action**.

Classificazione su clip e rilevazione su video lunghi

Il problema può quindi essere formulato in due modi complementari. Il primo riguarda la **classificazione temporale di clip video**: il modello riceve in input una clip già estratta e assegna una classe all'intera sequenza. Il secondo riguarda la **rilevazione di eventi su video lunghi**: il video completo viene analizzato tramite finestre temporali, le predizioni vengono filtrate e aggregate, e il sistema restituisce una sequenza di eventi con classe, tempo di inizio e tempo di fine.

Questa formulazione consente di separare il riconoscimento delle singole azioni dalla costruzione del report finale su partita intera.

3.2 Dataset e preprocessing

3.2.1 Acquisizione e annotazioni

Video sorgente

Il dataset è stato costruito a partire da video di partite giovanili di basket acquisiti tramite una **camera mobile automatica**, progettata per seguire lo sviluppo dell'azione e in particolare il movimento della palla. Tutti i video utilizzati per il dataset sono in formato **mp4** e hanno risoluzione **Full HD**, pari a 1920×1080 pixel. La maggior parte dei video sorgente è acquisita a circa 29.97 fps, mentre un solo video del dataset presenta un framerate pari a 24 fps.

Annotazioni temporali

Le annotazioni sono state realizzate tramite **ELAN**, definendo intervalli temporali precisi per le sette classi di azione: **passaggio**, **tiroDaDue0**, **tiroDaDue1**, **tiroDaTre0**, **tiroDaTre1**, **tiroLibero0** e **tiroLibero1**. I confini temporali sono stati definiti seguendo il protocollo di annotazione: il passaggio inizia quando la palla lascia le mani del giocatore, mentre i tiri terminano quando l'esito diventa osservabile.

Qualità delle riprese

Sebbene la camera segua automaticamente il gioco, le riprese non sono sempre ideali per il riconoscimento automatico. In alcuni casi il sistema di ripresa non inquadra correttamente l'azione, la palla esce temporaneamente dal campo visivo, oppure sono presenti occlusioni tra i giocatori. Per questo motivo sono state annotate solo le azioni **chiaramente riconoscibili**, evitando di introdurre label incerte.

3.2.2 Generazione delle clip e classi di background

Clip annotate

A partire dalle annotazioni sono state generate clip video associate alla rispettiva label. Oltre alle sette classi di azione, sono state considerate due classi di background: **non-gioco** e **idle**. La classe **non-gioco** è stata usata per indicare i segmenti in cui la partita non era effettivamente in corso, ad esempio durante pause, falli, interruzioni arbitrali o fasi di preparazione alla ripresa del gioco.

La classe **idle**, invece, è stata generata automaticamente campionando porzioni in cui il gioco era attivo, ma non era presente alcuna azione target, cioè nessun tiro o passaggio.

Poiché i confini temporali derivano direttamente dalle annotazioni, le clip generate non hanno una durata fissa, ma mantengono l'estensione temporale reale dell'evento annotato.

Unione di **idle** e **non-gioco**

Nelle prime fasi sperimentali **idle** e **non-gioco** sono state mantenute come classi distinte. Successivamente, poiché entrambe corrispondono a segmenti in cui il sistema non deve produrre annotazioni nel report finale, sono state unite nella classe funzionale **no-action**.

Questa scelta riduce la confusione tra due classi che hanno significato diverso dal punto di vista del dataset, ma svolgono lo stesso ruolo nell'inferenza finale: **evitare la generazione di eventi quando non è presente un'azione rilevante**.

Filtraggio delle clip

Durante la costruzione del dataset sono state inoltre rimosse le clip eccessivamente lunghe, in particolare quelle superiori a circa **5 secondi**. Questa scelta è stata introdotta per limitare lo sbilanciamento, soprattutto dovuto ai segmenti di background, e per rendere più omogenea la distribuzione temporale delle clip.

3.2.3 Split, caratteristiche tecniche e sbilanciamento

Suddivisione del dataset

Il dataset finale contiene **5713 clip**, suddivise in **4236 clip di training**, **540 di validation** e **937 di test**. Lo split è stato effettuato **per video**, e non per singola clip, in modo da evitare *data leakage* tra sequenze molto simili provenienti dallo stesso filmato.

Ogni clip è descritta nel file **manifest.csv**, che riporta il percorso del file, il video sorgente, i timestamp di inizio e fine, la label e lo split di appartenenza.

Sbilanciamento delle classi

Nonostante la rimozione delle clip più lunghe, il dataset rimane **fortemente sbilanciato**. La classe dominante è **passaggio**, con 2505 clip, pari a circa il 43.85% del dataset. Seguono le classi di background, con 1497 clip di **non-gioco** e 853 clip di **idle**. Le classi di tiro sono invece molto meno rappresentate: nel complesso costituiscono una porzione ridotta del dataset, con alcune categorie particolarmente rare, come **tiroDaTre1**, presente in soli 58 esempi, pari a circa l'1.02% delle clip totali.

Questo sbilanciamento ha influenzato le scelte successive di modellazione e valutazione. In particolare, ha motivato la **formulazione gerarchica del problema**, separando prima il riconoscimento tra azione e background e poi la classificazione più fine dei tiri.

3.3 Estrazione delle feature

Feature visuali pre-estratte

Per rappresentare il contenuto visuale delle clip è stato utilizzato un approccio basato su **feature pre-estratte** tramite **DINOv3**. Ogni clip viene letta frame per frame e ciascun frame viene convertito in un descrittore visuale mediante un backbone **pre-addestrato**. Negli esperimenti finali è stata utilizzata la variante `dinov3_vitl16`, che produce feature di dimensione **1024** per ogni frame.

Le feature estratte vengono salvate in file `.pt` e successivamente riutilizzate durante training, validazione e test. Questa scelta permette di separare la fase più costosa, cioè l'esecuzione del backbone visuale sui frame, dalla fase di addestramento dei classificatori temporali.

Motivazione dell'approccio

La scelta di utilizzare DINOv3 come **feature extractor congelato**, invece di effettuare il *fine-tuning* del backbone o di addestrare *end-to-end* un modello video completo, è motivata dalla dimensione limitata e dallo **sbilanciamento del dataset**. Aggiornare i pesi del backbone avrebbe aumentato sensibilmente il costo computazionale e il rischio di **overfitting**, soprattutto sulle classi meno rappresentate.

L'estrazione offline permette invece di calcolare una sola volta le rappresentazioni visuali e di riutilizzarle in più esperimenti. In questo modo la pipeline risulta più **modulare**, più efficiente dal punto di vista computazionale e più adatta al confronto rapido tra configurazioni diverse del classificatore temporale.

Preprocessing dei frame

Un aspetto importante riguarda il **preprocessing dei frame**. Inizialmente erano state utilizzate trasformazioni standard basate su `resize` e *center crop*. Tuttavia, da un'analisi visiva dei frame preprocessati è emerso che il *center crop* tendeva a rimuovere porzioni laterali dell'immagine.

Questo comportamento era problematico nel nostro scenario, poiché elementi rilevanti come giocatori, palla, canestro o parti del campo possono trovarsi anche ai bordi dell'inquadratura, soprattutto quando la camera automatica non segue correttamente l'azione.

Configurazione finale del preprocessing

Per evitare questa perdita di informazione, il preprocessing è stato modificato adottando un `resize` diretto dell'intero frame a dimensione quadrata, senza *center crop*. La configurazione finale utilizza quindi uno **stretching** a 336×336 pixel.

Successivamente, i valori dei pixel vengono normalizzati con le statistiche ImageNet, così da mantenere il preprocessing coerente con quello atteso dal backbone DINOv3 pre-addestrato. Questa scelta introduce una deformazione geometrica dell'immagine, ma consente di preservare tutto il contenuto visivo della scena.

Risoluzione di input

La risoluzione di input è stata inoltre aumentata rispetto alle prime prove sperimentali. Tale scelta comporta un maggiore **costo computazionale** nella fase di estrazione, ma permette al backbone di lavorare su immagini con più dettaglio.

Nel basket questo aspetto è particolarmente rilevante, perché oggetti discriminanti come la **palla**, il **canestro** e la posizione dei giocatori rispetto alla linea da tre punti possono occupare una porzione molto piccola del frame.

3.4 Tracking palla-canestro

Motivazione

Per arricchire le feature visuali estratte con DINOv3, è stato introdotto un modulo di **tracking palla-canestro**. L'obiettivo è fornire al classificatore informazioni esplicite sulla posizione e sul movimento della palla, oltre che sulla sua relazione geometrica con il canestro.

Nel basket, infatti, molte azioni non sono distinguibili solo dalla scena globale. Per riconoscere correttamente un tiro, un passaggio o l'esito di una conclusione è spesso necessario osservare la **traiettoria della palla** e la sua interazione con il ferro.

Detector YOLO

A tale scopo è stato addestrato un detector **YOLO11m** per rilevare due classi: **ball**, cioè la palla, e **rim**, cioè il ferro del canestro. Per addestrare questo detector sono stati costruiti dataset specifici a partire da frame estratti dalle clip del dataset principale e annotati manualmente in CVAT.

In una prima fase è stato creato un dataset composto esclusivamente da frame provenienti dalle clip di tiro dello split di training dell'action recognition, per un totale di **1183 frame**, poi suddivisi internamente per il training e la validazione del detector YOLO. Questa scelta era coerente con l'obiettivo iniziale di migliorare soprattutto la distinzione tra tiro segnato e tiro fallito, concentrandosi sui momenti finali della conclusione, in cui la palla si avvicina al canestro e l'esito diventa osservabile.

Fine-tuning del detector

A differenza del backbone DINOv3, utilizzato come feature extractor congelato, il detector **YOLO11m** è stato sottoposto a *fine-tuning* sul dataset palla-canestro costruito per il progetto. Il training parte da pesi pre-addestrati **yolo11m.pt** e aggiorna il detector sul task specifico di rilevamento delle due classi **ball** e **rim**.

Una volta addestrati, i detector YOLO vengono però utilizzati come moduli fissi per generare le detection palla-canestro e le successive feature geometriche e temporali. I loro pesi non vengono aggiornati durante il training dei classificatori temporali della gerarchia.

Dataset palla-canestro esteso

Successivamente è stato costruito un dataset più completo, includendo non solo frame di tiro, ma anche frame provenienti da clip di **passaggio**, **idle** e **non-gioco**. Il dataset esteso contiene **1663 immagini di training** e **320 immagini di validation**.

L'obiettivo di questa estensione era rendere il detector più robusto anche in contesti diversi dalla conclusione a canestro, dove la palla può trovarsi lontana dal ferro oppure non essere visibile. I frame di tiro sono stati estratti principalmente dalla parte finale della clip, mentre per **passaggio**, **idle** e **non-gioco** è stato usato un campionamento più uniforme lungo la durata della clip.

I frame senza palla o canestro visibile sono stati comunque mantenuti, poiché rappresentano esempi negativi utili per ridurre i falsi positivi del detector.

Estrazione delle detection

Per ogni clip del dataset di action recognition vengono campionati **fino a 48 frame** in modo uniforme lungo la durata della clip. Se la clip contiene più di 48 frame, ne vengono selezionati 48 distribuiti temporalmente; se invece la clip è più breve, vengono utilizzati tutti i frame disponibili.

Su ciascun frame il detector YOLO produce le bounding box di palla e canestro, quando visibili. A partire da queste detection vengono calcolate feature **geometriche** e **temporali**, tra cui la presenza o assenza degli oggetti, le coordinate normalizzate dei bounding box, la confidenza

delle detection, la distanza palla-canestro, la posizione relativa della palla rispetto al ferro e la velocità della palla nel tempo.

Feature temporali **temp29** e **temp43**

Le feature temporali sono state definite in due versioni principali. La prima, indicata come **temp29**, contiene **29 feature per frame** e descrive principalmente visibilità, posizione, dimensione degli oggetti, distanza palla-canestro, velocità e accelerazione della palla.

La seconda, indicata come **temp43**, contiene **43 feature per frame** ed estende la rappresentazione precedente aggiungendo segnali più specifici sull'interazione tra palla e canestro. Tra questi rientrano la sovrapposizione tra bounding box, l'ingresso del centro della palla nella regione del ferro, il movimento relativo palla-canestro e l'attraversamento verticale del livello del *rim*.

Ruolo del tracking nella classificazione

Queste informazioni sono utili in punti diversi della pipeline. Nel caso dei **passaggi**, il tracking aiuta a descrivere il movimento della palla e a distinguere una reale azione di gioco da segmenti di **no-action**. Nel caso dei **tiri**, la relazione tra palla e canestro fornisce indizi utili per distinguere la tipologia di conclusione e, soprattutto, l'esito del tiro.

Feature come la vicinanza al ferro, l'attraversamento della quota del *rim* e la presenza della palla all'interno della regione del canestro possono infatti aiutare il modello a separare i tiri segnati da quelli falliti.

Integrazione con le feature DINOv3

Le sequenze di tracking vengono salvate in formato NPZ, insieme a un indice JSON che associa ogni clip alla rispettiva sequenza temporale. Durante il training, tali sequenze vengono interpolate alla stessa lunghezza temporale delle feature DINOv3 della clip e concatenate frame per frame alle feature visuali.

Poiché le feature DINOv3-L/16 hanno dimensione **1024**, l'input del classificatore diventa di dimensione **1053** quando si usa **temp29** e di dimensione **1067** quando si usa **temp43**. In questo modo il modello riceve sia una rappresentazione globale della scena, sia una descrizione esplicita della geometria e del movimento palla-canestro.

Utilizzo nei livelli gerarchici

Le rappresentazioni di tracking sono state utilizzate in modo diverso nei livelli della pipeline gerarchica, selezionando per ciascun livello la combinazione più adatta tra detector YOLO e set di feature temporali.

In questa sezione si è descritto il processo di estrazione e rappresentazione delle feature palla-canestro; la scelta dei modelli finali impiegati nei singoli livelli e la selezione dei checkpoint vengono invece discusse nella sezione dedicata al training.

3.5 Classificatore gerarchico

Motivazione della gerarchia

Il riconoscimento delle azioni è stato formulato tramite un **classificatore gerarchico** composto da tre livelli distinti. L'idea alla base di questa scelta è decomporre il problema originale, caratterizzato da classi numerose e sbilanciate, in sottoproblemi più semplici e semanticamente coerenti.

Invece di addestrare un unico modello a distinguere direttamente tutte le classi finali, la pipeline separa prima le macro-categorie principali e successivamente specializza la classificazione solo sui casi in cui è necessario un maggiore dettaglio.

Livello L1

Il primo livello, indicato come **L1**, ha il compito di distinguere tra **passaggio**, **tiro** e **no-action**. In questo livello le classi **idle** e **non-gioco** vengono collassate in un'unica classe **no-action**, poiché entrambe rappresentano situazioni in cui non deve essere prodotta alcuna annotazione finale.

Questa scelta consente di ridurre la confusione tra due classi funzionalmente equivalenti per l'obiettivo del progetto e permette al modello di concentrarsi sulla distinzione tra **azioni rilevanti** e segmenti da scartare.

Routing dopo L1

Se L1 predice **passaggio**, la predizione finale viene assegnata direttamente alla classe **passaggio**. Se invece L1 predice **no-action**, la clip viene considerata come background e non viene associata a nessuna delle sette azioni finali.

Solo nel caso in cui L1 predica **tiro**, la clip viene inviata ai due livelli successivi della gerarchia.

Livello L2

Il secondo livello, indicato come **L2**, viene applicato soltanto alle clip classificate come tiro da L1. Il suo compito è riconoscere la **tipologia del tiro**, distinguendo tra **tiroDaDue**, **tiroDaTre** e **tiroLibero**.

In questo modo il modello non deve distinguere simultaneamente tra passaggi, **no-action** e tutte le varianti di tiro, ma lavora su un sottoinsieme più omogeneo di esempi, composto esclusivamente da conclusioni a canestro.

Livello L3

Il terzo livello, indicato come **L3**, viene anch'esso applicato solo alle clip che L1 ha classificato come tiro. Questo livello ha il compito di stimare l'**esito del tiro**, distinguendo tra **tiro0**, cioè tiro fallito, e **tiro1**, cioè tiro segnato.

La separazione tra tipo di tiro ed esito consente di trattare in modo indipendente due aspetti diversi dell'azione: da un lato la posizione o situazione di tiro, dall'altro l'interazione finale tra palla e canestro.

Ricostruzione della classe finale

Le predizioni di L2 e L3 vengono infine combinate per ricostruire la classe finale del tiro. Ad esempio, se L2 predice **tiroDaDue** e L3 predice **tiro1**, la classe finale diventa **tiroDaDue1**; analogamente, una predizione **tiroLibero** da L2 e **tiro0** da L3 produce la classe finale **tiroLibero0**.

In questo modo la gerarchia permette di ottenere le sei classi di tiro finali combinando una predizione relativa al **tipo di tiro** e una predizione relativa all'**esito**.

Input dei classificatori

Ogni livello della gerarchia utilizza un classificatore temporale applicato alla sequenza di feature della clip. L'input è costituito dalle feature DINOv3 *frame-level*, eventualmente concatenate con le feature di tracking palla-canestro descritte nella sezione precedente.

Il classificatore produce una distribuzione di probabilità sulle classi del livello considerato, e la classe con probabilità maggiore viene utilizzata per determinare il percorso della clip nella gerarchia.

Vantaggi della decomposizione

La decomposizione gerarchica è particolarmente adatta al dataset considerato per tre motivi principali. In primo luogo, riduce la complessità del problema rispetto a una classificazione

diretta sulle classi finali. In secondo luogo, permette di gestire meglio lo **sbilanciamento delle classi**, poiché i livelli L2 e L3 vengono addestrati su sottoinsiemi più specifici e semanticamente coerenti.

Infine, consente di integrare feature differenti nei punti in cui sono più utili: ad esempio, le informazioni palla-canestro sono particolarmente rilevanti nei livelli dedicati ai tiri e all'esito della conclusione.

3.6 Training

Training separato dei livelli

I tre livelli del classificatore gerarchico sono stati addestrati in modo **separato**, utilizzando sempre le feature DINOv3 pre-estratte come rappresentazione visuale di base. Questa scelta consente di ottimizzare ciascun livello sul proprio spazio di label, evitando che un unico modello debba apprendere contemporaneamente la distinzione tra azioni molto diverse e classi fortemente sbilanciate.

Spazio di label

Il training è stato organizzato definendo, per ciascun livello, un diverso spazio di classificazione. Nel caso di **L1**, le label originali vengono rimappate nelle tre classi **passaggio**, **tiro** e **no-action**. In questo livello tutte le classi di tiro vengono collassate nella macro-classe **tiro**, mentre **idle** e **non-gioco** vengono collassate in **no-action**.

Nel caso di **L2**, vengono mantenute solo le clip di tiro e le label vengono rimappate in **tiroDaDue**, **tiroDaTre** e **tiroLibero**. Infine, nel caso di **L3**, vengono usate solo le clip di tiro, ma le label vengono rimappate in **tiro0** e **tiro1**, a seconda dell'esito della conclusione.

Temporal Transformer

Ogni livello utilizza un **Temporal Transformer** come classificatore temporale. Il modello riceve in input una sequenza di feature *frame-level* e produce una predizione sulla classe del livello considerato.

La sequenza può essere composta solo dalle feature DINOv3 oppure dalle feature DINOv3 concatenate con le feature temporali di tracking palla-canestro. In quest'ultimo caso, le sequenze di tracking vengono portate alla stessa lunghezza temporale delle feature DINOv3 e concatenate frame per frame.

Gestione delle sequenze a lunghezza variabile

Poiché le clip hanno durata variabile, anche il numero di feature *frame-level* associate a ciascuna clip può cambiare. Durante il training, le sequenze vengono quindi organizzate in batch tramite padding temporale, mantenendo comunque l'informazione sulla lunghezza reale di ciascuna clip. In questo modo il **Temporal Transformer** può operare su sequenze di durata diversa senza imporre una lunghezza temporale unica a tutte le azioni.

Normalizzazione delle feature tracking

Durante il training, le feature di tracking vengono normalizzate tramite una normalizzazione **z-score** calcolata sui campioni del training set. Le statistiche di normalizzazione vengono poi riutilizzate in validazione, test e inferenza, così da mantenere coerente la distribuzione delle feature tra le diverse fasi della pipeline.

Gestione dello sbilanciamento

Per gestire lo **sbilanciamento delle classi**, sono state utilizzate due strategie complementari. La prima consiste nell'impiego di una **CrossEntropyLoss** **pesata**, in cui le classi meno rappresentate ricevono un peso maggiore. La seconda consiste nell'uso di un **WeightedRandomSampler**, che

aumenta la probabilità di campionare esempi appartenenti alle classi minoritarie durante il training.

Entrambe le strategie sono controllate da un parametro di potenza pari a **0.5**, in modo da compensare lo sbilanciamento senza rendere eccessivamente instabile l'ottimizzazione.

Ottimizzazione

L'ottimizzazione dei classificatori temporali è stata effettuata con **AdamW**, utilizzando il parametro *weight decay* per regolarizzare i pesi del modello e ridurre il rischio di **overfitting**. Durante la retropropagazione è stato inoltre applicato il *gradient clipping*, che limita la norma dei gradienti ed evita aggiornamenti troppo bruschi dei pesi, contribuendo a rendere il training più stabile.

È stato utilizzato anche lo scheduler **ReduceLRonPlateau**, configurato per monitorare la **Macro F1 sul validation set**. Quando tale metrica non migliora per un certo numero di epoche, il learning rate viene ridotto automaticamente, permettendo al modello di proseguire l'ottimizzazione con aggiornamenti più piccoli. Infine, per aumentare la riproducibilità degli esperimenti, è stato fissato un seed comune, utilizzato per inizializzazione, campionamento dei dati e operazioni casuali del training.

Iperparametri principali

La stessa configurazione di base è stata mantenuta per i tre livelli della gerarchia, modificando principalmente lo spazio di label e le feature tracking associate al livello considerato. I classificatori sono stati addestrati per un massimo di **100 epoche**, con **batch size 64** e learning rate iniziale pari a $5 \cdot 10^{-5}$.

Il Transformer temporale utilizza dimensione interna **256**, **2 layer**, **4 attention heads**, dimensione feed-forward pari a **768** e dropout pari a **0.45**. Come strategia di aggregazione temporale è stato utilizzato il *mean pooling*, che produce una rappresentazione complessiva della clip mediando le informazioni lungo la sequenza temporale.

Parametro	Valore
Epoche massime	100
Batch size	64
Learning rate	$5 \cdot 10^{-5}$
Ottimizzatore	AdamW
Weight decay	$5 \cdot 10^{-3}$
Dimensione feature DINOv3	1024
Dimensione interna Transformer	256
Numero layer Transformer	2
Numero attention heads	4
Dimensione feed-forward	768
Dropout	0.45
Pooling temporale	mean
Gradient clipping	1.0
Class weight power	0.5
Sampler power	0.5
Seed	42

Tabella 1: Principali iperparametri utilizzati per il training dei classificatori temporali.

Selezione dei checkpoint

La selezione del modello finale di ciascun livello avviene sulla base della **Macro F1 sul validation set**. Al termine di ogni epoca, il modello viene valutato sullo split di validazione; se la Macro

F1 migliora rispetto al miglior valore osservato fino a quel momento, viene salvato il checkpoint migliore.

Questa scelta permette di selezionare ciascun livello in modo indipendente e di ricostruire successivamente la pipeline gerarchica completa.

Configurazione finale dei livelli

La configurazione finale è stata ottenuta selezionando, per ciascun livello, la variante migliore osservata su validation. In particolare, **L1** utilizza feature tracking **temp43** estratte con il detector **YOLO v2**. Questo detector è stato addestrato sul dataset palla-canestro più completo, che include frame di tiro ma anche frame di **passaggio**, **idle** e **non-gioco**; per questo motivo risulta più adatto al primo livello, che deve operare su tutte le clip e distinguere tra azioni e background.

Anche **L2** utilizza il detector **YOLO v2**, ma con la rappresentazione **temp29**, che ha fornito risultati migliori per la classificazione della tipologia di tiro.

Scelta di YOLO v1 per L3

Per **L3** è stata invece mantenuta la rappresentazione **temp43** estratta con il detector **YOLO v1**. Questo detector corrisponde alla versione storica addestrata solo su frame provenienti dalle clip di tiro del training set, quindi su un dataset meno generale rispetto a quello usato per YOLO v2.

Nonostante questo limite, tale configurazione è risultata la più efficace per il livello dedicato all'esito del tiro, probabilmente perché il detector era specializzato proprio sui momenti finali delle conclusioni a canestro, in cui la relazione tra palla e ferro è più discriminante per distinguere **tiro0** e **tiro1**.

Separazione tra split e feature tracking

È importante sottolineare che questa differenza riguarda il detector usato per generare le feature di tracking, non la logica di split del classificatore gerarchico. I classificatori L1, L2 e L3 vengono comunque addestrati e validati secondo gli split del dataset di action recognition; ciò che cambia è la sorgente delle feature palla-canestro concatenate alle feature DINOv3.

La pipeline finale combina quindi i migliori checkpoint dei tre livelli, mantenendo separato il training ma integrando le predizioni nella fase di valutazione gerarchica.

Livello	Label space	Detector tracking	Feature	Input
L1	passaggio/tiro/no-action	YOLO v2 completo	temp43	1024 + 43
L2	tiroDaDue/tiroDaTre/tiroLibero	YOLO v2 completo	temp29	1024 + 29
L3	tiro0/tiro1	YOLO v1 shot-only	temp43	1024 + 43

Tabella 2: Configurazione finale dei tre livelli della gerarchia. YOLO v2 è addestrato sul dataset palla-canestro completo, mentre YOLO v1 è il detector storico addestrato solo su frame di tiro.

3.7 Inferenza su video lunghi

Obiettivo dell'inferenza long-video

Il modello descritto nelle sezioni precedenti è stato addestrato su clip temporali già segmentate. Per applicarlo a un video lungo è quindi necessario trasformare il filmato continuo in una sequenza di finestre temporali, classificare ciascuna finestra e convertire le predizioni locali in eventi temporali finali.

A tale scopo è stata sviluppata una pipeline di **inferenza su video lunghi**, progettata per riutilizzare la configurazione gerarchica finale senza generare fisicamente nuove clip video.

Feature frame-level del video

La prima fase consiste nella costruzione di una rappresentazione *frame-level* del video lungo. Il segmento temporale di interesse viene letto frame per frame e, per ciascun frame, vengono estratte le feature visuali tramite **DINOv3**, utilizzando lo stesso backbone e lo stesso preprocessing adottati nella pipeline su clip.

In questo modo si ottiene una sequenza temporale di descrittori visuali associati ai frame del video.

Detection palla-canestro

In parallelo, sugli stessi frame vengono eseguiti i detector YOLO palla-canestro. Poiché la configurazione finale utilizza **YOLO v2** per i livelli L1 e L2 e **YOLO v1** per il livello L3, vengono conservate le informazioni prodotte da entrambi i detector.

Per ogni frame vengono considerate le detection di **ball** e **rim**, mantenendo, in caso di più rilevazioni della stessa classe, quella con confidenza maggiore. A partire da queste informazioni vengono poi ricostruite le feature geometriche e temporali necessarie al tracking palla-canestro.

Sliding window multi-scala

A partire dalla sequenza di feature del video vengono generate finestre temporali virtuali tramite una strategia di *sliding window multi-scala*. Ogni finestra è definita da un tempo di inizio e un tempo di fine, e viene ricostruita direttamente a partire dalle feature *frame-level* già estratte.

Nella configurazione finale **exp_long_13** sono state utilizzate finestre di durata **1.0**, **1.5**, **2.0**, **2.5** e **3.0** secondi, con stride temporale pari a **0.25** secondi. L'uso di più durate consente di intercettare azioni con estensione temporale diversa, evitando di imporre una singola scala fissa a tutto il video.

Costruzione dell'input per finestra

Per ciascuna finestra, le feature DINOv3 vengono prese da tutti i frame appartenenti all'intervallo temporale considerato. Le feature di tracking palla-canestro vengono invece ricostruite a partire dalle detection YOLO salvate per i frame della finestra.

Se la finestra contiene più di **48 frame**, ne vengono selezionati al massimo 48 in modo uniforme; se ne contiene meno, vengono usati tutti i frame disponibili. Su questi frame vengono calcolate le informazioni geometriche e temporali relative a palla e canestro, come presenza degli oggetti, posizione relativa, distanza, movimento della palla e interazione con il ferro.

La sequenza di tracking ottenuta viene poi interpolata alla stessa lunghezza temporale della sequenza DINOv3 e normalizzata con le statistiche salvate nel checkpoint del livello corrispondente.

Feature usate dai livelli della gerarchia

La costruzione dell'input dipende dal livello della gerarchia. Per L1 vengono usate le feature tracking **temp43** prodotte a partire da **YOLO v2**; per L2 vengono usate le feature **temp29**, sempre basate su **YOLO v2**; per L3 vengono usate le feature **temp43** ricostruite a partire da **YOLO v1**.

In tutti i casi, le feature di tracking vengono concatenate frame per frame alle feature DINOv3, ottenendo una rappresentazione temporale coerente con quella utilizzata durante il training dei tre livelli.

Combinazione probabilistica delle uscite

Ogni finestra viene classificata utilizzando i tre livelli della gerarchia finale. Nella pipeline long-video, tuttavia, la gerarchia non viene applicata come un routing rigido basato solo sulla predizione di L1. Per ogni finestra vengono calcolate le probabilità di tutti e tre i livelli: L1 produce le probabilità di **passaggio**, **tiro** e **no-action**; L2 produce le probabilità di **tiroDaDue**, **tiroDaTre** e **tiroLibero**; L3 produce le probabilità dell'esito 0 o 1.

Gli score delle classi finali vengono quindi ottenuti combinando probabilisticamente le uscite dei tre livelli. Lo score di **passaggio** coincide con la probabilità assegnata da L1 alla classe **passaggio**, mentre lo score di **no-action** coincide con la probabilità assegnata da L1 alla classe **no-action**.

Per le classi di tiro, invece, lo score finale è ottenuto moltiplicando la probabilità di **tiro** prodotta da L1, la probabilità del tipo di tiro prodotta da L2 e la probabilità dell'esito prodotta da L3. Ad esempio:

$$P(\text{tiroDaDue1}) = P_{L1}(\text{tiro}) \cdot P_{L2}(\text{tiroDaDue}) \cdot P_{L3}(1).$$

La label assegnata alla finestra è quindi la classe finale con score massimo.

Post-processing temporale

Le predizioni sulle finestre non possono essere usate direttamente come annotazioni, poiché finestre vicine e sovrapposte possono generare più predizioni per la stessa azione. Per questo motivo viene applicato un **post-processing deterministico**, il cui obiettivo è trasformare le predizioni locali in una lista più compatta di eventi temporali.

Innanzitutto vengono scartate tutte le finestre classificate come **no-action**. Le finestre rimanenti vengono considerate candidate solo se superano una soglia minima di confidenza, pari a **0.75** per **passaggio** e **0.40** per le classi di tiro. Inoltre, per ridurre i falsi positivi, lo score dell'azione deve essere sufficientemente maggiore dello score di **no-action**; nella configurazione finale è stato utilizzato un margine pari a **20.0**.

Vincoli sulle finestre candidate

Sono stati applicati anche vincoli sulla durata delle finestre candidate. Le finestre classificate come **passaggio** vengono accettate solo se hanno durata non superiore a **2.0 secondi**; con le scale temporali utilizzate, questo corrisponde alle finestre da **1.0**, **1.5** e **2.0** secondi.

Le finestre classificate come tiro vengono invece accettate solo se hanno durata compresa tra **1.0** e **3.0 secondi**, quindi possono usare tutte le scale generate: **1.0**, **1.5**, **2.0**, **2.5** e **3.0** secondi.

Generazione degli eventi finali

Le finestre candidate appartenenti alla stessa classe vengono raggruppate se temporalmente vicine, utilizzando un gap massimo di **0.20 secondi**. Se il distacco tra due finestre candidate consecutive supera tale valore, il gruppo corrente viene chiuso e viene avviato un nuovo gruppo. In questo modo, predizioni separate da un intervallo temporale sufficientemente ampio vengono trasformate in eventi distinti.

Ogni gruppo viene poi trasformato in uno o più eventi. Gli estremi temporali sono ricavati dagli estremi delle finestre che compongono il gruppo, mentre la confidence dell'evento viene calcolata come il massimo delle confidence delle finestre del gruppo. Per evitare la generazione di annotazioni eccessivamente lunghe, i gruppi continui vengono eventualmente suddivisi imponendo una durata massima pari a **1.50 secondi** per **passaggio** e a **3.00 secondi** per le classi di tiro. Questa suddivisione può produrre eventi consecutivi senza necessariamente introdurre un intervallo vuoto tra essi.

Vengono inoltre scartati gli eventi con durata inferiore a **0.70 secondi**.

Gestione degli overlap

Infine, la configurazione finale non consente eventi sovrapposti. Nel caso in cui due eventi si sovrappongano temporalmente, viene mantenuto l'evento con confidence più alta e l'altro viene scartato.

Il risultato finale è una lista di eventi contenente, per ciascuna azione rilevata, la label predetta, il tempo di inizio, il tempo di fine, la durata, la confidence e il numero di finestre che hanno contribuito alla sua generazione.

Parametro	Valore
Durate sliding window	1.0, 1.5, 2.0, 2.5, 3.0 s
Stride sliding window	0.25 s
Frame tracking massimi per finestra	48
Finestra massima passaggio	2.00 s
Finestra minima tiro	1.00 s
Finestra massima tiro	3.00 s
Soglia passaggio	0.75
Soglia tiri	0.40
Margine action/no-action	20.0
Gap massimo per merge	0.20 s
Durata minima evento	0.70 s
Durata massima evento passaggio	1.50 s
Durata massima evento tiro	3.00 s
Confidence evento	massimo delle finestre
Overlap tra eventi	non consentito

Tabella 3: Principali parametri utilizzati nella pipeline di inferenza e post-processing su video lunghi.

Output della pipeline

Questa pipeline consente quindi di passare da un video continuo a una lista compatta di eventi annotati temporalmente. Le predizioni finali possono essere utilizzate sia per una valutazione *event-level*, confrontandole con le annotazioni temporali disponibili, sia per generare una preview video con overlay delle azioni riconosciute.

Nel caso di video privi di annotazioni complete, la stessa pipeline produce comunque un report temporale degli eventi rilevati e una visualizzazione qualitativa utile per analizzare il comportamento del modello.

3.8 Aspetti implementativi

Organizzazione modulare

La pipeline sviluppata è stata organizzata in modo **modulare**, separando le fasi di estrazione delle feature, training, valutazione, inferenza su video lunghi, post-processing e generazione della preview annotata. Questa struttura ha permesso di modificare e valutare singole componenti senza dover ripetere ogni volta l'intero processo.

Feature precomutate

Un aspetto centrale è l'uso di **feature precomutate**. Le feature DINOv3 vengono estratte offline una sola volta e poi riutilizzate durante training, validazione e test. In questo modo l'esecuzione del backbone visuale sui frame viene separata dall'addestramento dei classificatori temporali.

Una volta calcolate le feature, è possibile addestrare e confrontare più configurazioni del **Temporal Transformer** senza rieseguire ogni volta la feature extraction.

Riutilizzo delle detection YOLO

Lo stesso principio è stato applicato alle informazioni palla-canestro prodotte dai detector YOLO. Le detection vengono riutilizzate per costruire le feature di tracking richieste dai diversi livelli della gerarchia, permettendo di confrontare rappresentazioni diverse, come **temp29** e **temp43**, e di selezionare configurazioni differenti per L1, L2 e L3.

Finestre virtuali nei video lunghi

Anche l'inferenza su video lunghi è stata progettata con un'impostazione analoga. Invece di generare molte clip intermedie, il video viene rappresentato tramite feature *frame-level* e le finestre temporali vengono costruite in modo virtuale.

Ogni finestra viene quindi classificata leggendo direttamente le feature già disponibili, riducendo operazioni ridondanti e semplificando l'applicazione del modello a segmenti video estesi.

Vantaggi pratici

Nel complesso, questa organizzazione ha reso più rapido il confronto tra configurazioni diverse e ha facilitato il riutilizzo delle componenti migliori nella pipeline finale. La separazione tra i moduli ha inoltre reso più semplice analizzare gli errori e intervenire in modo mirato sulle singole fasi del sistema.

4 Esperimenti e risultati

4.1 Protocollo sperimentale

Gli esperimenti sono stati valutati su validation e test utilizzando **accuracy**, **Macro F1** e **Weighted F1**. La **Macro F1** è stata considerata particolarmente importante a causa dello sbilanciamento tra le classi, poiché **passaggio** e le classi di background sono molto più frequenti rispetto alle classi di tiro.

Nelle prime fasi sperimentali, le classi **idle** e **non-gioco** sono state mantenute separate, coerentemente con la struttura originale del dataset. Tuttavia, durante gli esperimenti è emerso che il modello **tendeva a confonderle** e che, dal punto di vista applicativo, entrambe avevano la stessa funzione: indicare segmenti in cui non deve essere generata alcuna annotazione nel report finale. Per questo motivo, nelle configurazioni successive, **idle** e **non-gioco** sono state **raggruppate nella classe no-action**. Questa scelta riduce una distinzione poco utile per l'obiettivo finale e permette al modello di concentrarsi sulla separazione tra azioni reali e segmenti da ignorare.

Sono state quindi considerate sia le **8 classi finali**, cioè le 7 azioni annotate più **no-action**, sia le sole **7 azioni di interesse**. La valutazione a 8 classi misura la capacità del modello di distinguere anche i segmenti senza azione, mentre la valutazione a 7 azioni misura il comportamento sulle classi che devono effettivamente comparire nel report finale. Per l'inferenza su video lunghi, invece, la valutazione è stata svolta a livello di evento, confrontando gli intervalli predetti con quelli annotati tramite **IoU temporale**.

4.2 Selezione del modello

Baseline e scelta del classificatore temporale

Gli esperimenti iniziali hanno confrontato modelli basati su feature visuali pre-estratte e classificatori temporali diversi. Le prime baseline con CNN pre-addestrate e GRU/BiGRU hanno mostrato risultati discreti sulle classi più frequenti, ma una maggiore difficoltà sulle classi rare, soprattutto sui tiri. Ad esempio, **exp_09**, basato su ConvNeXt-Tiny e BiGRU, raggiungeva Accuracy 0.7400 e Macro F1 0.4600 su validation.

Per migliorare la modellazione temporale è stato quindi introdotto un **Temporal Transformer Encoder**. Questa scelta è stata motivata dalla necessità di catturare relazioni tra frame anche non consecutivi, aspetto importante nelle azioni cestistiche, dove il significato dell'azione dipende spesso dall'evoluzione del movimento nel tempo. Con ConvNeXt-Tiny, il passaggio al Transformer non ha prodotto un miglioramento netto rispetto alla BiGRU, ma ha fornito una base più flessibile per sfruttare feature visuali più robuste.

Bilanciamento delle classi

Poiché il dataset è fortemente sbilanciato, sono state testate diverse strategie basate su **loss pesata** e **WeightedRandomSampler**. Nelle prime prove con BiGRU, l'uso congiunto di loss pesata e sampler non ha migliorato stabilmente le prestazioni: **exp_07** otteneva Macro F1 0.4200, mentre **exp_08**, con solo sampler, raggiungeva 0.4400 e **exp_09**, con sola loss pesata, arrivava a 0.4600. Questo ha mostrato che un bilanciamento troppo forte poteva rendere il training meno stabile.

Con il Temporal Transformer è stato quindi adottato un bilanciamento più moderato. Senza sampler, **exp_11** raggiungeva Accuracy 0.7389 ma Macro F1 0.4046, indicando una maggiore tendenza a privilegiare le classi frequenti. Reintroducendo un sampler moderato, **exp_13** otteneva Macro F1 0.4557 e Weighted F1 0.7187. Per questo motivo, nella fase di selezione del modello su feature visuali sono stati mantenuti valori moderati, in particolare **class_weight_power=0.5** e **sampler_power=0.4**.

Feature visuali e dimensione del Transformer

Il miglioramento più evidente è arrivato cambiando il feature extractor. Passando da ConvNeXt-Tiny a DINOv2 e poi a DINOv3, le prestazioni sono aumentate in modo significativo, confermando l'importanza di utilizzare rappresentazioni visuali più robuste. Con DINOv2-L/14 sono state confrontate anche configurazioni del **Temporal Transformer** con dimensione interna diversa. Le configurazioni a **d_model=256**, come **exp_17** e **exp_18**, ottenevano Macro F1 rispettivamente pari a 0.5749 e 0.5762. Aumentando la capacità del classificatore temporale a **d_model=384**, **exp_19** raggiungeva Accuracy 0.8167 e Weighted F1 0.8079, migliorando le metriche globali rispetto alle configurazioni precedenti, pur con una Macro F1 leggermente più bassa, pari a 0.5672.

Questa differenza ha suggerito che un Transformer più capiente non massimizzava necessariamente la Macro F1, ma forniva un comportamento più solido in termini di Accuracy e Weighted F1. Per questo motivo si è scelto di proseguire con la configurazione a **d_model=384**, considerata più adatta come base operativa per gli esperimenti successivi.

Con DINOv3-L/16 le prestazioni sono migliorate ulteriormente. **exp_21**, con *CLS pooling*, otteneva la Macro F1 più alta tra gli esperimenti preliminari, pari a 0.6256. Tuttavia, **exp_22**, con *mean pooling*, raggiungeva Accuracy 0.8296 e Weighted F1 0.8232, migliori rispetto a **exp_21**, mantenendo comunque una Macro F1 elevata pari a 0.6146. Per questo motivo è stata scelta la configurazione con **DINOv3-L/16**, **Temporal Transformer a d_model=384** e **mean pooling**, considerata il miglior compromesso tra accuratezza complessiva, stabilità e comportamento operativo.

Data augmentation

È stata testata anche una **data augmentation mirata sulle classi di tiro**, con l'obiettivo di aumentare la robustezza sulle classi meno rappresentate. Le augmentation sono state applicate generando una versione aumentata delle feature DINOv3, indicata come **dinov3_vit16_336_aug_shots**. In particolare, sono state usate due trasformazioni leggere sulle clip di tiro: **flip orizzontale** e **color jitter**.

Il **flip orizzontale** consiste nel ribaltamento sinistra-destra dei frame della clip. Questa trasformazione mantiene invariata la classe dell'azione, perché un tiro da due, da tre o libero conserva lo stesso significato anche se osservato dal lato opposto del campo. L'obiettivo era rendere il modello meno dipendente dalla posizione dell'azione nell'inquadratura e dalla direzione del movimento.

Il **color jitter** introduce invece variazioni fotometriche controllate sui frame. In particolare, sono stati modificati **luminosità**, **contrasto**, **saturazione** e **tonalità**, con parametri rispettivamente pari a **brightness=0.2**, **contrast=0.2**, **saturation=0.2** e **hue=0.05**. Queste trasformazioni simulano variazioni moderate di illuminazione, esposizione e resa cromatica tra video diversi, senza alterare il contenuto semantico dell'azione.

Tuttavia, rispetto a **exp_22**, le configurazioni con augmentation non hanno migliorato le metriche globali. **exp_23** ha ottenuto Accuracy 0.8185, Macro F1 0.6025 e Weighted F1 0.8081, mentre **exp_24**, con bilanciamento più leggero, ha raggiunto Accuracy 0.8241, Macro F1 0.5941 e Weighted F1 0.8102. L’augmentation ha portato alcuni miglioramenti locali su singole classi di tiro, ma non un vantaggio complessivo sufficiente. Per questo motivo non è stata mantenuta nella configurazione selezionata.

Configurazione selezionata

Alla fine della fase di selezione preliminare, **exp_22** è stata scelta come riferimento tra gli esperimenti basati sulle sole feature visuali: **DINOv3-L/16 frozen** come estrattore di feature, **Temporal Transformer Encoder**, *mean pooling*, loss pesata e sampler moderato. Questa configurazione non massimizza ogni singola metrica, ma offre il miglior compromesso operativo tra Accuracy, Macro F1 e Weighted F1 nella fase senza tracking.

Gli esperimenti successivi hanno quindi mantenuto la combinazione **DINOv3 + Temporal Transformer + mean pooling** come base metodologica, introducendo però la formulazione gerarchica e le feature di tracking palla-canestro. In questa fase finale, i livelli della gerarchia sono stati addestrati con la configurazione riportata nella metodologia, utilizzando una dimensione interna del Transformer pari a **d_model=256**.

Esperimento	Configurazione	Accuracy	Macro F1	Weighted F1
exp_09	ConvNeXt-Tiny + BiGRU, sola loss pesata	0.7400	0.4600	0.7300
exp_13	ConvNeXt-Tiny + Temporal Transformer, sampler moderato	0.7167	0.4557	0.7187
exp_19	DINOv2-L/14 + Transformer d_model=384	0.8167	0.5672	0.8079
exp_21	DINOv3-L/16 + Transformer, CLS pooling	0.8148	0.6256	0.8095
exp_22	DINOv3-L/16 + Transformer, mean pooling	0.8296	0.6146	0.8232
exp_23	DINOv3-L/16 + Transformer, augmentation sui tiri	0.8185	0.6025	0.8081
exp_24	DINOv3-L/16 + Transformer, augmentation e bilanciamento leggero	0.8241	0.5941	0.8102

Tabella 4: Esperimenti preliminari rappresentativi su validation.

4.3 Gerarchia e tracking palla-canestro

Motivazione della gerarchia

Dopo la selezione del modello basato su **DINOv3** e **Temporal Transformer**, è stata valutata la formulazione del problema. Nelle prime configurazioni il modello lavorava sulle classi originali, mantenendo separate anche **idle** e **non-gioco**; successivamente queste due classi sono state collassate in **no-action**, poiché entrambe rappresentano segmenti da non riportare nel report finale. Anche con questa semplificazione, la **classificazione piatta rimaneva complessa**, perché il modello doveva distinguere contemporaneamente background, passaggi, tipo di tiro ed esito del tiro. Inoltre, alcune classi di tiro erano poco rappresentate, rendendo la sola accuracy poco informativa.

Un ulteriore vantaggio della formulazione gerarchica riguarda lo **sbilanciamento tra le classi**. Nel problema piatto, il modello deve confrontare direttamente una classe molto frequente come **passaggio** con classi molto rare, ad esempio **tiroDaTre1**. Nella gerarchia, invece, il primo livello distingue **passaggio**, **tiro** e **no-action**, dove tutti i tiri vengono inizialmente raggruppati in un’unica macro-classe. In questo modo la differenza tra **passaggio** e l’insieme dei tiri è meno estrema rispetto alla differenza tra **passaggio** e una singola sottoclasse rara di tiro. I livelli successivi lavorano poi su sottoproblemi più specifici: L2 distingue il tipo di tiro solo tra le clip di tiro, mentre L3 distingue l’esito aggregando i tiri in **tiro0** e **tiro1**. Questo rende ciascuna decisione più omogenea e riduce l’effetto dello sbilanciamento sulle classi finali.

Per questo motivo sono state sperimentate diverse rimappature delle label, con l'obiettivo di capire quali sottoproblemi fossero più facilmente separabili. Gli esperimenti di *label remapping* hanno mostrato che il modello **distingue bene le macro-categorie** passaggio, tiro e no-action: **exp_28** raggiunge Accuracy 0.9074 e Macro F1 0.9073. Anche la classificazione del tipo di tiro risulta abbastanza robusta, con **exp_29** che ottiene Macro F1 0.8410. Al contrario, la **distinzione dell'esito del tiro è più difficile**: **exp_30**, dedicato a tiro0/tiro1, raggiunge Macro F1 0.6425. Questi risultati hanno motivato una **pipeline gerarchica in tre livelli**: **L1** distingue **passaggio, tiro e no-action**; **L2** classifica il tipo di tiro; **L3** classifica l'esito del tiro. La classe finale viene ricostruita combinando le uscite di L2 e L3.

Tracking palla-canestro

Dopo aver definito la gerarchia, è stato introdotto il **tracking palla-canestro** per fornire al modello informazioni esplicite sulla posizione della palla, sul suo movimento e sulla relazione con il ferro. Sono state considerate tre versioni del detector YOLO. **YOLO v1** è stato addestrato inizialmente su frame estratti da clip di tiro, concentrandosi sui momenti finali dell'azione, dove l'esito diventa osservabile. **YOLO v2** è stato addestrato su un dataset più esteso, includendo anche frame di **passaggio, idle e non-gioco**, così da gestire contesti più generali. **YOLO v3** è stato invece usato come ulteriore confronto *shot-only* con split di validation più controllato.

Da questi detector sono state estratte feature di tracking in forma aggregata e temporale. Le feature **aggregate39** rappresentano ogni clip tramite 39 statistiche globali sulla traiettoria della palla e sulla relazione palla-canestro, ad esempio posizione, distanza dal ferro e variazioni complessive nel tempo. Poiché sono feature aggregate a livello di clip, lo stesso vettore viene replicato lungo la sequenza temporale e concatenato alle feature DINOv3 di ogni frame.

Le feature **temp29**, invece, mantengono una rappresentazione temporale *frame-level*: per ogni istante descrivono la geometria osservata della palla e del canestro, permettendo al Transformer di analizzare l'evoluzione del movimento nel tempo. Le feature **temp43** sono una versione estesa delle temporali, con 43 descrittori per frame, e includono informazioni geometriche aggiuntive per rendere più ricca la descrizione della dinamica palla-canestro. In entrambi i casi, le sequenze di tracking vengono interpolate alla stessa lunghezza delle feature DINOv3 e concatenate frame per frame.

Le tre versioni YOLO sono state testate sui tre livelli della gerarchia. La **configurazione migliore** è risultata **YOLO v2 temp43** per L1, **YOLO v2 temp29** per L2 e **YOLO v1 temp43** per L3.

Confronto finale e scelta della configurazione

Prima di fissare la configurazione finale è stato effettuato un **confronto tra modello gerarchico e modello non gerarchico** usando tracking aggregato. Questo confronto serviva a verificare se l'aggiunta delle feature palla-canestro fosse sufficiente per rendere competitivo un unico classificatore diretto sulle classi finali, oppure se la decomposizione gerarchica continuasse a essere vantaggiosa. Il modello gerarchico con tracking aggregato su L3, **exp_38**, ottiene Macro F1 0.6996 sulle 8 classi finali, mentre il modello non gerarchico **exp_39** ottiene Macro F1 0.5620. Il miglioramento conferma che la **gerarchia gestisce meglio lo sbilanciamento** e la diversa natura delle decisioni da prendere. Con il passaggio al tracking temporale su L3, **exp_40** sale a Macro F1 0.7223, mentre la **configurazione finale exp_46**, con il tracking migliore per ciascun livello, raggiunge Macro F1 0.8079.

Esperimento	Configurazione	Accuracy	Macro F1	Weighted F1
exp_28	L1: passaggio /tiro/no-action	0.9074	0.9073	0.9072
exp_29	L2: tipo di tiro solo sulle clip di tiro	0.8462	0.8410	0.8520
exp_30	L3: esito del tiro solo sulle clip di tiro	0.6615	0.6425	0.6615
exp_38	Gerarchico, DINOv3 + YOLO v1 aggregate 39 su L3	0.8759	0.6996	0.8766
exp_39	Non gerarchico, DINOv3 + YOLO v1 aggregate 39	0.8426	0.5620	0.8447
exp_40	Gerarchico, DINOv3 + YOLO v1 temp 29 su L3	0.8815	0.7223	0.8808
exp_46	Gerarchico, migliore tracking per ciascun livello	0.9056	0.8079	0.9062

Tabella 5: Esperimenti che motivano la scelta della formulazione gerarchica e del tracking palla-canestro. Le metriche sono calcolate su validation. Gli esperimenti **exp_28**, **exp_29** e **exp_30** valutano singoli sottoproblemi, mentre **exp_38**, **exp_39**, **exp_40** ed **exp_46** sono valutati end-to-end sulle 8 classi finali.

4.4 Inferenza su video lunghi

Pipeline di inferenza

La valutazione su clip misura la capacità del modello di classificare segmenti già estratti e centrati sull'azione. Tuttavia, l'obiettivo finale del progetto richiede di applicare il sistema a video completi, producendo un report temporale delle azioni rilevate. Per questo motivo è stata sviluppata una pipeline di inferenza su video lunghi basata su *sliding window* e *post-processing temporale*.

Il video viene analizzato tramite finestre temporali sovrapposte. Ogni finestra viene classificata dal modello gerarchico e produce una classe con una confidenza. Le predizioni **no-action** vengono scartate, mentre le predizioni di azione vengono aggregate temporalmente. Il post-processing ha il compito di unire finestre vicine appartenenti alla stessa classe, eliminare duplicati, gestire sovrapposizioni e filtrare eventi poco affidabili. Questa fase è necessaria perché una singola azione può essere coperta da più finestre consecutive, mentre alcune finestre di gioco attivo possono generare falsi positivi, soprattutto sulla classe **passaggio**.

Evoluzione degli esperimenti e scelta della configurazione

Gli esperimenti long-video hanno mostrato che il problema principale non era soltanto classificare correttamente le singole finestre, ma **trasformare molte predizioni sovrapposte in un insieme di eventi pulito e credibile**. Le prime configurazioni erano molto permissive: **exp_long_01**, ad esempio, usava molte scale temporali e uno stride fitto, producendo 225 eventi. Questa scelta permetteva di ottenere una recall elevata, pari a 0.7606, ma generava 171 falsi positivi e una precision molto bassa, pari a 0.2400. Il risultato indicava che il modello intercettava molte azioni reali, ma la preview risultava troppo rumorosa.

La **prima svolta** è arrivata con l'uso di **finestre più lunghe per i passaggi**. L'idea era fornire più contesto temporale al modello, evitando che finestre molto brevi venissero classificate come **passaggio** anche in fasi di transizione o di gioco attivo senza azioni annotate. Con **exp_long_04** il numero di predizioni scende a 97 e la F1 sale a 0.5238, mostrando che un **contesto temporale più ampio riduce sensibilmente i falsi positivi**. La variante **exp_long_05**, con durata massima dei passaggi ridotta, recupera qualche vero positivo in più, ma aumenta anche i falsi positivi; per questo non rappresenta un miglioramento netto.

Successivamente è stata testata una **strategia peak-based** per i passaggi, negli esperimenti **exp_long_06**, **exp_long_07** e **exp_long_08**. In questo caso l'obiettivo era separare la classificazione dalla localizzazione: il modello classificava finestre più lunghe, ma l'evento finale veniva centrato attorno ai picchi di confidenza. Questa strategia ha migliorato in alcuni casi la localizzazione temporale, ma non ha migliorato abbastanza la detection complessiva. Le

configurazioni più selettive perdevano troppi eventi reali, mentre quelle più permissive tornavano a generare troppi falsi positivi.

Un'altra **svolta importante** è stata l'introduzione di un **filtro più severo** rispetto alla classe **no-action**. Con `exp_long_09`, l'uso di un margine *action/no-action* più forte riduce il rumore e porta la F1 a 0.5500, con 89 eventi predetti. Questo esperimento ha mostrato che la **distinzione tra azione e background è decisiva** per rendere la pipeline utilizzabile su video lunghi. Tuttavia, un filtro troppo aggressivo, come in `exp_long_10`, aumenta la precision fino a 0.5441 ma riduce eccessivamente la recall a 0.5211, perdendo troppe azioni reali.

La configurazione `exp_long_13` nasce quindi come **compromesso tra queste osservazioni**: mantiene finestre più lunghe, usa un margine **no-action** intermedio e produce un numero di eventi più realistico. Con 80 predizioni, 42 veri positivi, 38 falsi positivi e 29 falsi negativi, ottiene precision 0.5250, recall 0.5915 e F1 0.5563, risultando il **miglior compromesso quantitativo** sulla validation. La scelta non dipende quindi da una singola metrica isolata, ma dal **bilanciamento tra copertura delle azioni reali, riduzione dei falsi eventi e qualità della preview finale**.

Infine, gli esperimenti `exp_long_15`–`exp_long_18` hanno analizzato il problema dei tiri. La prova con sovrapposizioni ammesse, `exp_long_15`, ha mostrato che molti tiri erano presenti nelle predizioni raw, ma venivano spesso eliminati durante la soppressione degli overlap o sovrastati dai passaggi. Per questo motivo `exp_long_16` e `exp_long_18` hanno introdotto una **priorità controllata ai tiri**. Queste varianti migliorano la visibilità dei tiri nella demo, ma **non superano `exp_long_13` come risultato globale**: in particolare, `exp_long_18` ottiene F1 0.5405, inferiore a `exp_long_13`. Per questo motivo `exp_long_13` è stata mantenuta come configurazione finale della pipeline *long-video*.

Esperimento	Pred	TP	FP	FN	Precision	Recall	F1
<code>exp_long_01</code>	225	54	171	17	0.2400	0.7606	0.3649
<code>exp_long_09</code>	89	44	45	27	0.4944	0.6197	0.5500
<code>exp_long_10</code>	68	37	31	34	0.5441	0.5211	0.5324
<code>exp_long_13</code>	80	42	38	29	0.5250	0.5915	0.5563
<code>exp_long_18</code>	77	40	37	31	0.5195	0.5634	0.5405

Tabella 6: Esperimenti rappresentativi sulla pipeline *long-video* nel segmento di validation.

Differenza rispetto alla classificazione su clip

Rispetto alla classificazione su clip, le metriche *long-video* sono più basse perché il sistema deve risolvere anche un **problema di localizzazione temporale**. Una predizione può avere classe corretta ma intervallo temporale non sufficientemente sovrapposto alla ground truth, oppure più finestre vicine possono generare eventi duplicati. Inoltre, nei video lunghi sono presenti molti segmenti di gioco attivo non annotati, che possono risultare visivamente simili a passaggi o preparazioni al tiro. Questo rende la fase di post-processing determinante per ottenere un report finale utilizzabile.

4.5 Discussione dei risultati

Risultati clip-level

Il modello finale, indicato come `exp_46`, raggiunge **buone prestazioni** nella classificazione *clip-level*. Su validation ottiene Accuracy 0.9056, Macro F1 0.8079 e Weighted F1 0.9062 sulle 8 classi finali. Sul *test set* ottiene Accuracy 0.8549, Macro F1 0.6749 e Weighted F1 0.8564. Il **calo della Macro F1 sul test** indica che il modello generalizza bene sulle classi frequenti, ma risulta più sensibile alle classi meno rappresentate e alle variazioni tra video.

Split	Acc. 8	Macro F1 8	Weighted F1 8	Acc. 7	Macro F1 7	Weighted F1 7
Validation	0.9056	0.8079	0.9062	0.9206	0.7919	0.8936
Test	0.8549	0.6749	0.8564	0.8619	0.6494	0.8577

Tabella 7: Risultati finali di `exp_46` su validation e test.

La valutazione sulle 8 classi include le 7 azioni annotate più `no-action`, mentre quella sulle 7 azioni considera solo le classi che devono comparire nel report finale. La **differenza tra Weighted F1 e Macro F1 conferma l'effetto dello sbilanciamento**: le classi più frequenti, come `passaggio` e `no-action`, vengono riconosciute in modo stabile, mentre le classi con pochi esempi incidono maggiormente sulla Macro F1.

Effetto dell'esito del tiro

Per capire quanto l'esito del tiro influenzi il risultato finale, è stata valutata anche una versione collassata in cui si ignora la distinzione tra tiro segnato e tiro sbagliato. In questa valutazione, ad esempio, `tiroDaDue0` e `tiroDaDue1` vengono entrambi ricondotti a `tiroDaDue`. I **risultati migliorano in modo evidente**: la Macro F1 passa da 0.8079 a 0.8744 su validation e da 0.6749 a 0.7898 su test. Questo mostra che il modello **riconosce in modo più robusto il tipo generale di azione** rispetto alla classificazione completa con esito separato.

Split	Accuracy tipo azione	Macro F1 tipo azione	Weighted F1 tipo azione
Validation	0.9111	0.8744	0.9111
Test	0.8762	0.7898	0.8784

Tabella 8: Valutazione collassata senza esito del tiro.

Analisi dei livelli gerarchici

Le metriche diagnostiche sui livelli della gerarchia confermano la **validità della decomposizione del problema**. Su validation, L1 raggiunge Macro F1 0.9264 nella distinzione tra `passaggio`, `tiro` e `no-action`. Per L2 e L3 sono disponibili sia le metriche condizionate nella gerarchia, calcolate sui veri tiri che arrivano effettivamente al ramo `tiro`, sia le metriche standalone, calcolate dando direttamente in input tutti i veri tiri del *validation set*. Le **metriche standalone** permettono di valutare il singolo livello senza dipendere dagli errori di L1.

Livello	Valutazione validation	Accuracy	Macro F1	Weighted F1
L1	Tutte le clip	0.9222	0.9264	0.9222
L2	Condizionata: veri tiri arrivati al ramo <code>tiro</code>	0.9032	0.9033	0.9032
L2	Standalone: tutti i veri tiri	0.9077	0.9095	0.9077
L3	Condizionata: veri tiri arrivati al ramo <code>tiro</code>	0.9516	0.9494	0.9521
L3	Standalone: tutti i veri tiri	0.9385	0.9366	0.9391

Tabella 9: Metriche diagnostiche dei livelli gerarchici su validation. Per L2 e L3 sono riportate sia le metriche condizionate nella gerarchia sia le metriche standalone sui veri tiri.

Sul *test set*, invece, la valutazione diagnostica disponibile considera L1 su tutte le clip e, per L2 e L3, i veri tiri che arrivano effettivamente al ramo `tiro` dopo L1. In questo caso **L1 rimane stabile**, con Macro F1 0.9090, mentre L2 scende a Macro F1 0.7607 e L3 ottiene Macro F1 0.8410. Il **livello più critico sul test è quindi L2**, cioè la distinzione tra `tiroDaDue`, `tiroDaTre` e `tiroLibero`.

Livello	Valutazione test	Accuracy	Macro F1	Weighted F1
L1	Tutte le clip	0.9082	0.9090	0.9079
L2	Condizionata: veri tiri arrivati al ramo tiro	0.8013	0.7607	0.8183
L3	Condizionata: veri tiri arrivati al ramo tiro	0.8411	0.8410	0.8414

Tabella 10: Metriche diagnostiche dei livelli gerarchici sul *test set*. Per L2 e L3 sono riportate le metriche disponibili sui veri tiri che arrivano al ramo **tiro** dopo L1.

Comportamenti corretti e criticità

Sul *validation set* il modello risulta **stabile sulle principali macro-categorie**: **passaggio** raggiunge F1-score 0.9203 e **no-action** F1-score 0.9194. Anche le classi di tiro ottengono risultati complessivamente buoni, sebbene alcune metriche vadano interpretate con cautela a causa del supporto ridotto, come nel caso di **tiroDaTre1**. Questo suggerisce che, sulle clip già ritagliate, la criticità principale non sia tanto il riconoscimento dei tiri, quanto la separazione tra azioni reali e segmenti da ignorare.

La **confusione più rilevante** riguarda infatti **passaggio** e **no-action**: su validation 25 campioni **no-action** vengono classificati come **passaggio**, mentre 9 passaggi vengono classificati come **no-action**; sul test i valori corrispondenti sono 42 e 28. Dal **punto di vista applicativo**, questo errore è importante perché può generare falsi eventi nella preview del video lungo oppure far perdere passaggi reali. Sul test emerge anche una **maggiore difficoltà nella classificazione fine dei tiri**: **tiroLibero0** e **tiroLibero1** ottengono F1-score pari rispettivamente a 0.4571 e 0.4706, e L2 risulta il livello più debole della gerarchia.

GT \ Pred	pass.	2pt0	2pt1	3pt0	3pt1	TL0	TL1	no-act.
passaggio	202	0	0	1	0	0	0	9
tiroDaDue0	0	18	2	0	0	1	0	0
tiroDaDue1	0	0	9	0	1	0	1	0
tiroDaTre0	0	0	0	10	1	0	0	1
tiroDaTre1	0	0	1	0	2	0	0	0
tiroLibero0	0	0	0	0	0	7	0	0
tiroLibero1	0	0	2	0	0	0	7	2
no-action	25	2	1	0	0	1	0	234

Tabella 11: Matrice di confusione di **exp_46** sul *validation set*, sulle 8 classi finali. Le righe rappresentano le classi reali, le colonne le predizioni. La classe **no-action** deriva dal raggruppamento di **idle** e **non-gioco**.

GT \ Pred	pass.	2pt0	2pt1	3pt0	3pt1	TL0	TL1	no-act.
passaggio	473	5	0	0	0	0	0	28
tiroDaDue0	3	27	11	4	0	2	0	1
tiroDaDue1	0	1	35	1	1	2	10	0
tiroDaTre0	0	1	0	16	4	7	1	0
tiroDaTre1	0	0	0	0	7	0	1	1
tiroLibero0	0	0	0	0	0	8	3	2
tiroLibero1	0	0	0	0	0	1	8	2
no-action	42	0	0	0	0	2	0	227

Tabella 12: Matrice di confusione di **exp_46** sul *test set*, sulle 8 classi finali. Le righe rappresentano le classi reali, le colonne le predizioni.

Risultati event-level su video lungo

Dopo la selezione della configurazione **exp_long_13** sulla validation, la stessa configurazione è stata valutata anche sul video di test. In entrambi i casi la valutazione è stata eseguita su un segmento di **10 minuti**: per la validation sul video **PrimaParte.mp4**, nell'intervallo 135s–735s, con 71 eventi annotati; per il test sul video **PSA_converted.mp4**, nell'intervallo 10s–610s, con

123 eventi annotati. Gli eventi predetti sono stati confrontati con quelli annotati usando una soglia di **IoU temporale** pari a 0.20.

Sulla validation, **exp_long_13** produce 80 eventi, con 42 veri positivi, 38 falsi positivi e 29 falsi negativi, ottenendo precision 0.5250, recall 0.5915 e F1 0.5563. Sul test, la stessa configurazione produce 164 eventi, con 69 veri positivi, 95 falsi positivi e 54 falsi negativi, ottenendo precision 0.4207, recall 0.5610 e F1 0.4808.

Split	Config.	GT	Pred	TP	FP	FN	Precision	Recall	F1
Validation	exp_long_13	71	80	42	38	29	0.5250	0.5915	0.5563
Test	test_exp_long_13	123	164	69	95	54	0.4207	0.5610	0.4808

Tabella 13: Risultati event-level della configurazione **exp_long_13** su segmenti di 10 minuti di validation e test. La F1 è calcolata a livello evento a partire dai conteggi globali di TP, FP e FN.

La configurazione **exp_long_13** viene quindi mantenuta come configurazione finale per l’inferenza su video lunghi. Pur producendo ancora diversi falsi positivi, soprattutto sul test, rappresenta il **miglior compromesso ottenuto** tra copertura delle azioni reali e numero di eventi generati. Questo conferma che, nel passaggio da clip ritagliate a video lunghi, la **difficoltà principale non è solo classificare correttamente le azioni**, ma trasformare predizioni su finestre sovrapposte in eventi temporalmente coerenti.

Interpretazione complessiva

Nel complesso, i risultati mostrano che la combinazione di **DINOv3**, **Temporal Transformer**, **gerarchia** e **tracking palla-canestro** è efficace per riconoscere le macro-categorie e molte delle azioni annotate. Il modello è **solido nella separazione** tra **passaggio**, **tiro** e **no-action**, e la valutazione senza esito del tiro conferma che il tipo generale di azione viene riconosciuto meglio della classe finale completa. Il **calo di prestazioni tra validation e test**, soprattutto in termini di Macro F1, suggerisce però una maggiore sensibilità alle variazioni tra video; questo effetto risulta più evidente sulle classi meno frequenti, per le quali pochi errori incidono maggiormente sulle metriche.

Rimangono **due limiti principali**. Il primo è la **confusione tra passaggio e no-action**, particolarmente rilevante perché influenza direttamente la qualità del report finale e della preview. Il secondo riguarda la **classificazione fine dei tiri**, che emerge soprattutto sul *test set* e nei livelli dedicati alla distinzione tra tipologie di tiro. Nei video lunghi, questi errori vengono amplificati dalla necessità di localizzare temporalmente gli eventi, rimuovere duplicati e filtrare falsi positivi. Miglioramenti futuri potrebbero quindi riguardare sia un **tracking più robusto della palla**, sia un **post-processing più adattivo** per separare meglio passaggi reali, tiri e fasi di gioco attivo senza azioni annotate.

5 Conclusioni

I risultati ottenuti mostrano che una pipeline basata su **feature visuali pre-addestrate**, **modellazione temporale** e **informazioni contestuali palla-canestro** può rappresentare una soluzione efficace per il riconoscimento di azioni cestistiche in un dataset custom di dimensioni limitate e caratterizzato da forte sbilanciamento tra le classi. La combinazione di **DINOv3**, **Temporal Transformer**, **classificazione gerarchica** e **tracking palla-canestro** ha permesso di ottenere un sistema modulare, applicabile sia alla classificazione di clip annotate sia all’inferenza su segmenti video più lunghi.

Risultati principali e inferenza su video lunghi

La configurazione finale **exp_46** raggiunge risultati **molto buoni** considerando la natura del dataset, composto da video reali, classi sbilanciate, annotazioni non sempre uniformi e azioni

spesso brevi o visivamente ambigue. Nella valutazione *clip-level*, il modello ottiene su *validation* Accuracy 0.9056 e Macro F1 0.8079, mentre sul *test set* raggiunge Accuracy 0.8549 e Macro F1 0.6749. Il **calo tra validation e test**, soprattutto in termini di Macro F1, suggerisce una certa sensibilità alle variazioni tra video e alla limitata disponibilità di esempi per alcune classi meno frequenti.

La formulazione **gerarchica** si è rivelata utile per decomporre il problema in decisioni più semplici e semanticamente coerenti, separando prima le macro-categorie e affrontando poi la classificazione più fine dei tiri. Le principali difficoltà rimangono infatti legate alla distinzione tra alcune tipologie di conclusione e alla separazione tra **passaggio** e **no-action**, specialmente nei casi in cui il gioco è attivo ma non è presente un'azione annotata.

L'applicazione a video lunghi ha confermato che il passaggio dalle clip ritagliate alla generazione di eventi temporalmente localizzati introduce una difficoltà aggiuntiva. Anche se la pipeline *long-video* presenta ancora alcune criticità, i risultati sono **complessivamente incoraggianti** considerando che l'approccio parte da un modello addestrato su clip già segmentate e viene poi adattato a finestre temporali sovrapposte su video continui. Gli errori osservati, in particolare falsi positivi e mancate rilevazioni, sono probabilmente influenzati anche dalla natura del dataset, dalla presenza di annotazioni parziali e dalla complessità delle fasi di gioco non annotate.

Limiti e sviluppi futuri

I principali limiti emersi riguardano la **confusione tra passaggio e no-action**, la **classificazione fine dei tiri** e la sensibilità dell'inferenza su video lunghi alla localizzazione temporale degli eventi. Questi aspetti incidono direttamente sulla qualità della preview finale, poiché possono generare eventi non presenti nelle annotazioni oppure causare la mancata rilevazione di azioni reali.

Sviluppi futuri potrebbero riguardare innanzitutto l'**ampliamento del dataset**, con particolare attenzione alle classi di tiro meno rappresentate e a una maggiore varietà di situazioni di gioco. Un miglioramento rilevante potrebbe inoltre derivare da un tracking più robusto non solo di **palla** e **canestro**, ma anche dei **giocatori** e di elementi geometrici del **campo**, così da fornire al modello informazioni più complete sul contesto dell'azione. Infine, una possibile direzione futura consiste nello sviluppare un approccio progettato direttamente per il **video lungo**, invece di partire da clip già ritagliate, integrando in modo più esplicito classificazione e localizzazione temporale degli eventi.

Riferimenti bibliografici

- [1] S. Francia, S. Calderara, and D. F. Lanzi, "Classificazione di azioni cestistiche mediante tecniche di deep learning," 2018, thesis/report available on ResearchGate.
- [2] S. R. Shakya, C. Zhang, and Z. Zhou, "Basketball-51: A video dataset for activity recognition in the basketball game," in *CS & IT Conference Proceedings*, vol. 11, 2021, pp. 141–153.
- [3] Z. Han, W. Qin, Y. Wang, Q. Wang, and Y. Shi, "Multisubjects: A multi-subject video dataset for single-person basketball action recognition from basketball gym," *Computer Vision and Image Understanding*, vol. 249, p. 104193, 2024.
- [4] D. Tran, L. Bourdev, R. Fergus, L. Torresani, and M. Paluri, "Learning spatiotemporal features with 3d convolutional networks," in *Proceedings of the IEEE International Conference on Computer Vision*, 2015, pp. 4489–4497.
- [5] J. Carreira and A. Zisserman, "Quo vadis, action recognition? a new model and the kinetics dataset," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 4724–4733.

- [6] K. Hara, H. Kataoka, and Y. Satoh, “Can spatiotemporal 3d cnns retrace the history of 2d cnns and imagenet?” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 6546–6555.
- [7] D. Tran, H. Wang, L. Torresani, J. Ray, Y. LeCun, and M. Paluri, “A closer look at spatiotemporal convolutions for action recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 6450–6459.
- [8] C. Feichtenhofer, H. Fan, J. Malik, and K. He, “Slowfast networks for video recognition,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 6202–6211.
- [9] A. Arnab, M. Dehghani, G. Heigold, C. Sun, M. Lučić, and C. Schmid, “Vivit: A video vision transformer,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 6836–6846.
- [10] G. Bertasius, H. Wang, and L. Torresani, “Is space-time attention all you need for video understanding?” in *Proceedings of the 38th International Conference on Machine Learning*, ser. *Proceedings of Machine Learning Research*, vol. 139. PMLR, 2021, pp. 813–824.
- [11] Z. Liu, J. Ning, Y. Cao, Y. Wei, Z. Zhang, S. Lin, and H. Hu, “Video swin transformer,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 3202–3211.
- [12] S. Yan, Y. Xiong, and D. Lin, “Spatial temporal graph convolutional networks for skeleton-based action recognition,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [13] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [14] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A convnet for the 2020s,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 11 976–11 986.
- [15] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, M. Assran, N. Ballas, W. Galuba, R. Howes, P.-Y. Huang, S.-W. Li, I. Misra, M. Rabbat, V. Sharma, G. Synnaeve, H. Xu, H. Jégou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski, “Dinov2: Learning robust visual features without supervision,” 2023.
- [16] O. Siméoni, H. V. Vo, M. Seitzer, F. Baldassarre, M. Oquab, C. Jose, V. Khalidov, M. Szafraniec, S. Yi, M. Ramamonjisoa, F. Massa, D. Haziza, L. Wehrstedt, J. Wang, T. Darcet, T. Moutakanni, L. Sentana, C. Roberts, A. Vedaldi, J. Tolan, J. Brandt, C. Couprie, J. Mairal, H. Jégou, P. Labatut, and P. Bojanowski, “Dinov3,” 2025.
- [17] M. Tan and Q. V. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” in *Proceedings of the 36th International Conference on Machine Learning*, ser. *Proceedings of Machine Learning Research*, vol. 97. PMLR, 2019, pp. 6105–6114.
- [18] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, “Learning transferable visual models from natural language supervision,” in *Proceedings of the 38th International Conference on Machine Learning*, ser. *Proceedings of Machine Learning Research*, vol. 139. PMLR, 2021, pp. 8748–8763.

- [19] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, “Masked autoencoders are scalable vision learners,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 16 000–16 009.
- [20] Z. Tong, Y. Song, J. Wang, and L. Wang, “Videomae: Masked autoencoders are data-efficient learners for self-supervised video pre-training,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 10 078–10 093.
- [21] Y. Zhang, F. Zhang, Y. Zhou, and X. Xu, “Aca-net: Adaptive context-aware network for basketball action recognition,” *Frontiers in Neurorobotics*, vol. 18, p. 1471327, 2024.